

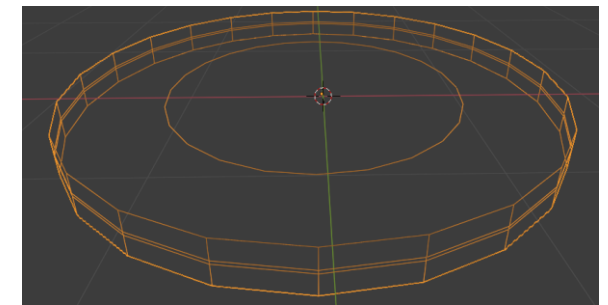
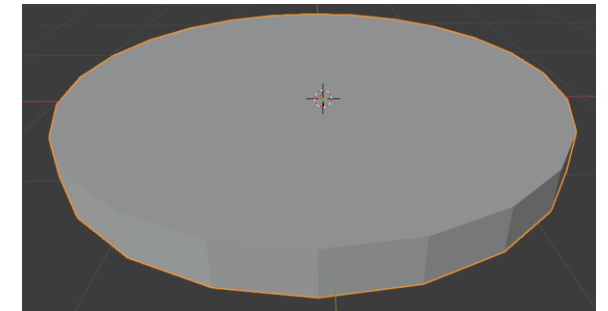
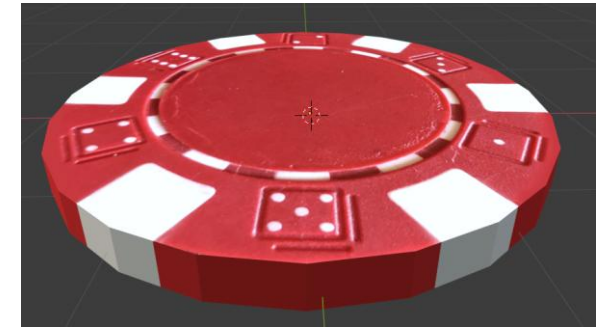
Video Game Programming: Lights and Materials

L07

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

3D models

- A 3D scene is made by one or more 3D models
- A 3D model is made of one or more **meshes**
- A mesh consist of:
 - **Vertices** (points with x, y, z coordinates)
 - **Edges** connecting pairs of vertices
 - **Faces** identified by 3 or more edges
 - **Polygons** made of coplanar faces
- Without additional information, 3D models have no color (usually rendered gray)



Render pipelines

- Rendering allows to see a 3D scene on a 2D screen by considering models, lights, and materials
- A render pipeline specifies how to perform:
 - Culling, i.e., deciding which faces will not be visible in the rendered image
 - Rendering, i.e., generating a 2D image from the 3D scene
 - Post-processing, i.e., applying effects to the generated image
- A game engine can offer different render pipelines for different goals and platforms, and some of these pipelines can be customized
- Unity has
 - a built-in render pipeline (the old default, soon deprecated),
 - a universal render pipeline for performance scalability,
 - and a high-definition render pipeline for higher fidelity

Rendering paths

- Pipelines can support different rendering paths, which determine lighting and shading operations
- Rendering path can be set for the project and for each camera
- Forward rendering path is the default, it works on all GPUs, but it is not suited if several real-time lights are needed
- Deferred rendering path works on some GPUs, it supports several real-time lights, but may have some limitations (no semi-transparent objects, no orthographic projection, no hardware anti-aliasing,...)

Lights

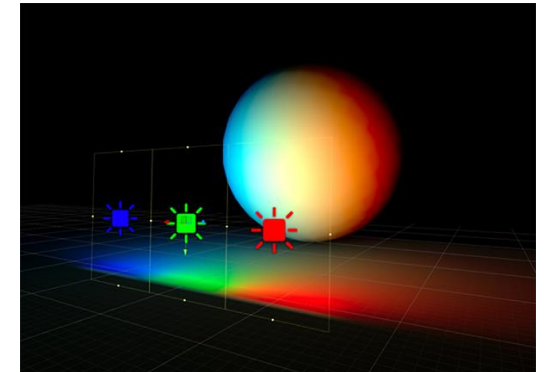
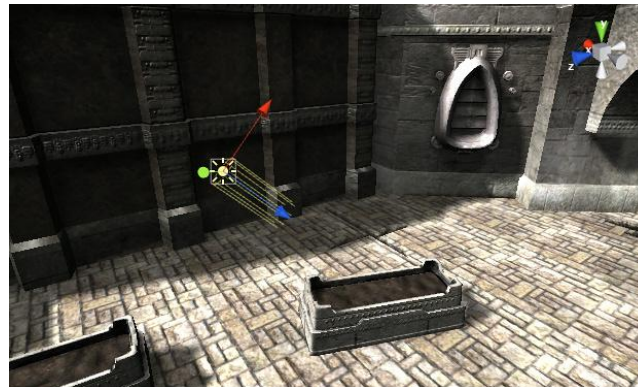
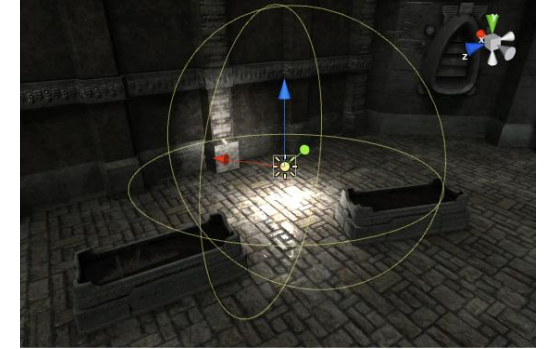
- Lights in game engines try to simulate real-world light properties
- **Direct** light is emitted, hits a surface, and is reflected directly into a sensor (e.g., eye or camera), while **indirect** light bounces on two or more surfaces before reaching the sensor
- **Real-time** lights are calculated while playing, while **baked** lights are calculated before building the game (for non-moving objects)
- **Global illumination** is a set of techniques that model direct, indirect, real-time, and baked lights also supporting reflection and light probes, but the support varies with render pipelines
- Light sources include actual lights, emissive materials, and ambient light

Light properties

- Supported light properties vary with game engine and render pipeline, but generally include:
 - Type, e.g., directional, spot, ...
 - Range, i.e., how far the light can travel
 - Spot angle, i.e., the angle of the cone in a spot light
 - Color, i.e., the color of the light
 - Mode, i.e., if it is computed in real-time, is baked or both
 - Intensity, i.e., how bright is the light
 - Indirect multiplier, i.e., how the lights dimmers after bouncing

Types of light

- Spot (top left) simulates a spotlight
- Point (top right) simulates a spherical light bulb
- Directional (bottom left) simulates the sun
- Area (bottom right) simulates a neon light

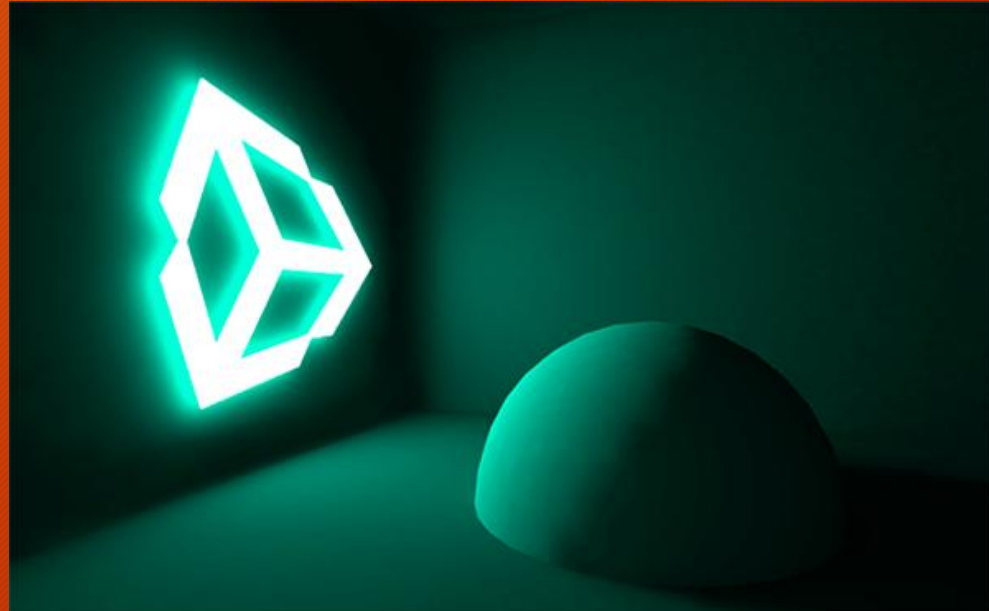


Shadows

- 3D models illuminated by lights may (or may not) cast shadows on themselves or on other 3D models
- Shadows are useful to understand position and scale
- Shadows can be calculated in real-time or baked (for non-moving objects)
- Shadow **strength** determines darkness, while **resolution** determines quality and performance cost
- Options to cast shadows can be set per light and per mesh
- Several shadow quality options can be set for the project

Emissive materials

- Meshes with emissive materials emit light across their surface, similarly to area light
- In Unity, light from emissive materials falloffs at a predefined rate and illuminates only static objects



Ambient light

- Ambient or environmental light is a base light that is present in the whole scene and has no source
- Very useful for cartoon-style graphics and to add some light uniformly in the scene
- Since it has no source and no direction, it generates no shadows

Design with lighting

- Lighting (and post-effects) might change a 3D environment
- The same scene with different lighting can look inviting or scary
- Lights can evoke emotions such as sadness, fear, and comfort
- Lights can be used to draw attention to specific parts of the scene

Materials

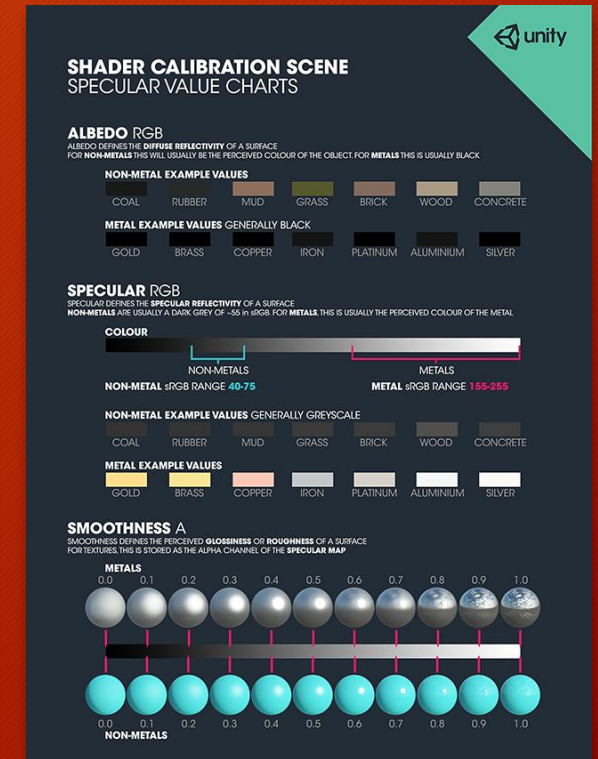
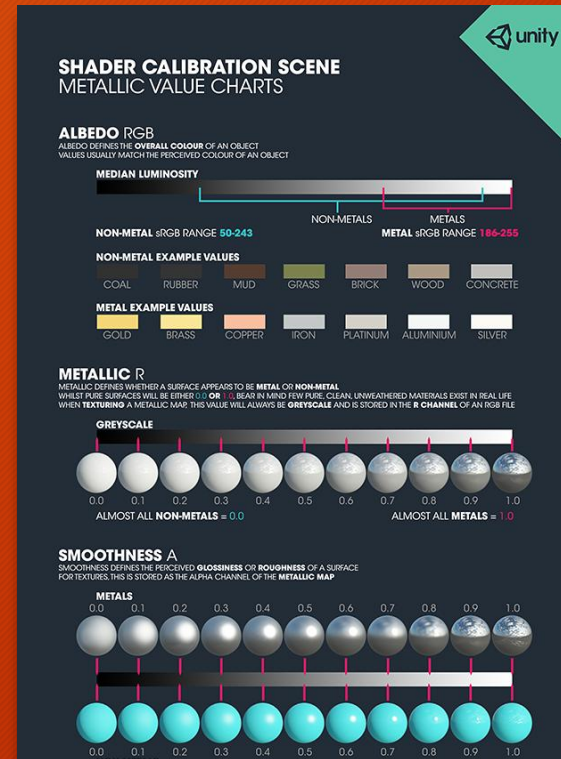
- To render colored 2D images from a scene with 3D models and lights, specific programs called **shaders** are used
- Materials specify how the 3D model should appear by setting the shaders to use for the different polygons and their options
- Depending on shader complexity, materials can allow to specify one or more colors for diffuse, specular, emissive behavior
- Textures are 2D bitmap images whose color information can be used for the color of the 3D model or for other aspects
- UV coordinates can be specified in the mesh to associate different parts of the 2D texture with different polygons

Physically based shading

- Physically Based Shading (PBS) simulates the interactions between materials and light in the real world
- PBS was not possible in real time until ten years ago
- PBS can simulate physics principles like energy conservation, Fresnel reflections, surface self-occlusion
- Unity Standard Shader implements PBS, combines different shaders, and is optimized considering the specified parameters

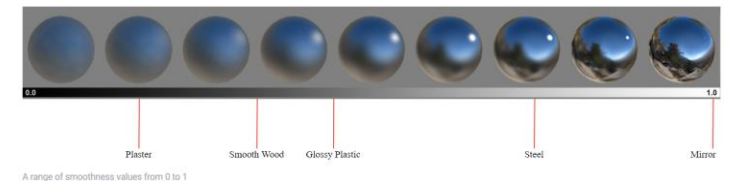
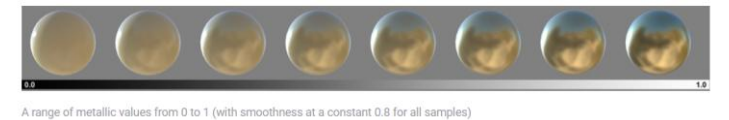
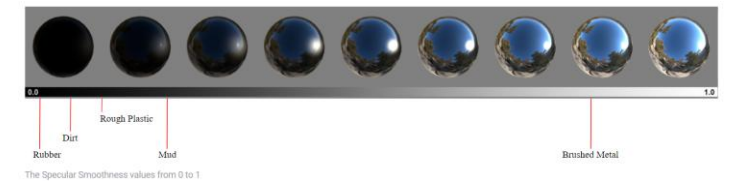
Metallic and specular setups

- Metallic setup allows to specify how much the material is metallic
- In metallic materials, specular reflections are of diffuse color
- In non-metallic materials, specular reflections are of light color
- Specular setup allows to choose a specular color to control color and strength of specular reflections



Material properties

- Albedo controls the base color of the surface usually with a texture whose alpha can be used to determine transparency
- Specular and metallic determines reflection properties
- Normal (bump) and height maps add details to the model
- Occlusion maps specify how the model receives indirect light
- Emission determines light emitted from the model
- Secondary maps add details over the main ones



Runtime editing of materials

- In Unity, material properties can be edited by code
- Properties of Material class such as color and mainTexture set diffuse properties of material
- Methods such as SetColor and SetTexture allow to specify which color or texture should be changed (e.g., "_EmissionColor")
- Since the Standard Shader is optimized, it is necessary to enable the variants that apply aspects to change
- Changing properties of a material generates a new instance unless shared material is used to change all objects using the material

Bibliografy

- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th ed., Packt Publishing, 2024
- Unity, Lighting up your graphics, 2020, <https://create.unity3d.com/real-time-lighting-for-games-in-unity>

